

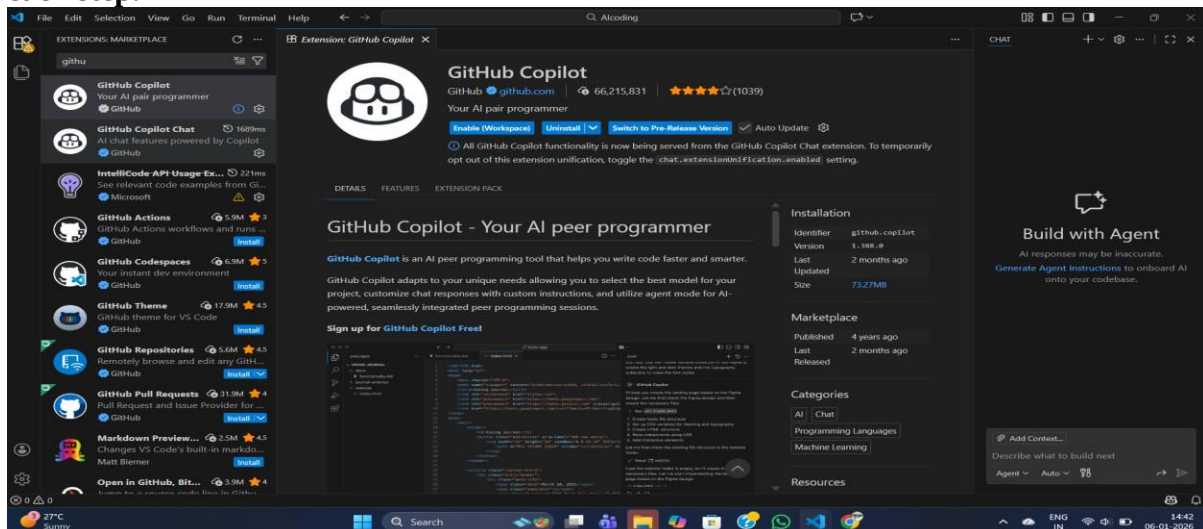
Lab Assignment # 1

Program : B. Tech (CSE)
Specialization :
Course Title : AI Assisted coding
Course Code :
Semester II
Academic Session : 2025-2026
Name of Student : A VamshiKrishna
Enrollment No. : 2403A51L47
Batch No. 52
Date :06-01-2026

Submission Starts here

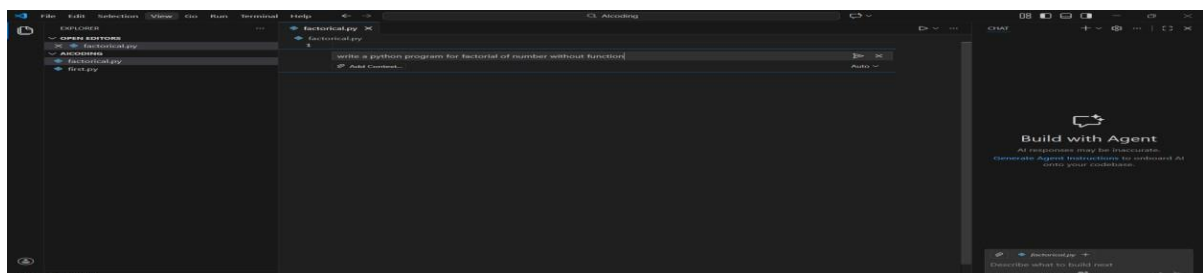
OUTPUT :
SCREENSHOTS:

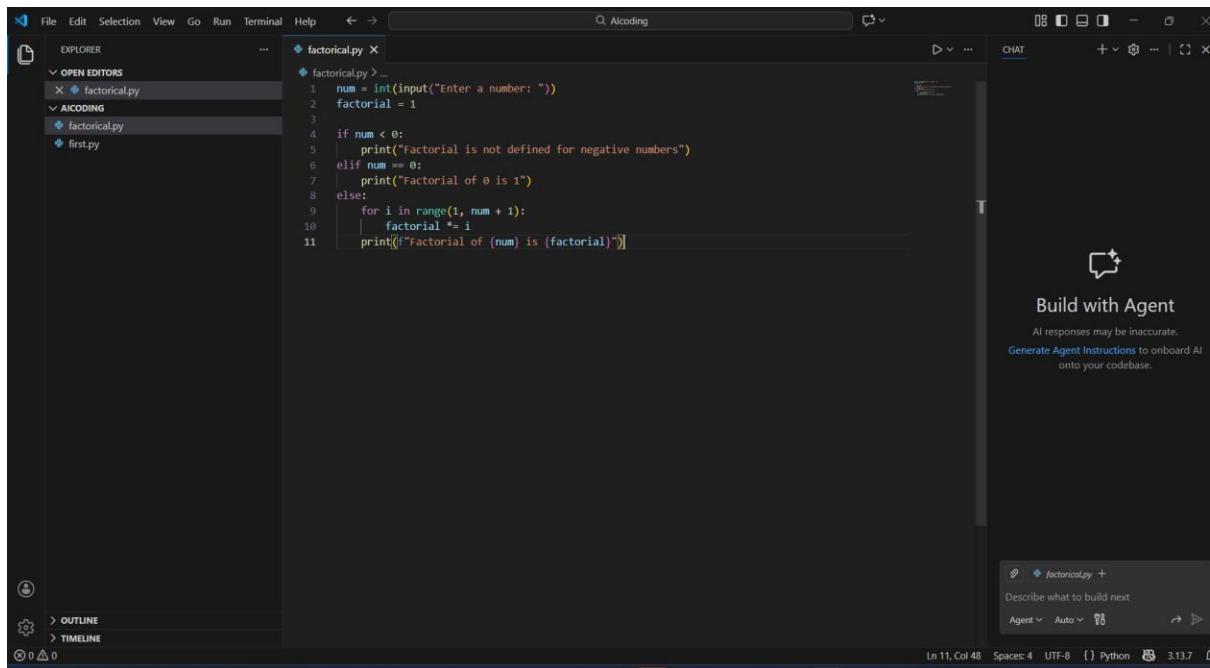
Task 0: Install and configure GitHub Copilot in VS Code. Take screenshots of each step.



Task1: Task Description

Use GitHub Copilot to generate a Python program that computes a mathematical product-based value (factorial-like logic) directly in the main execution flow, without using any user-defined functions.

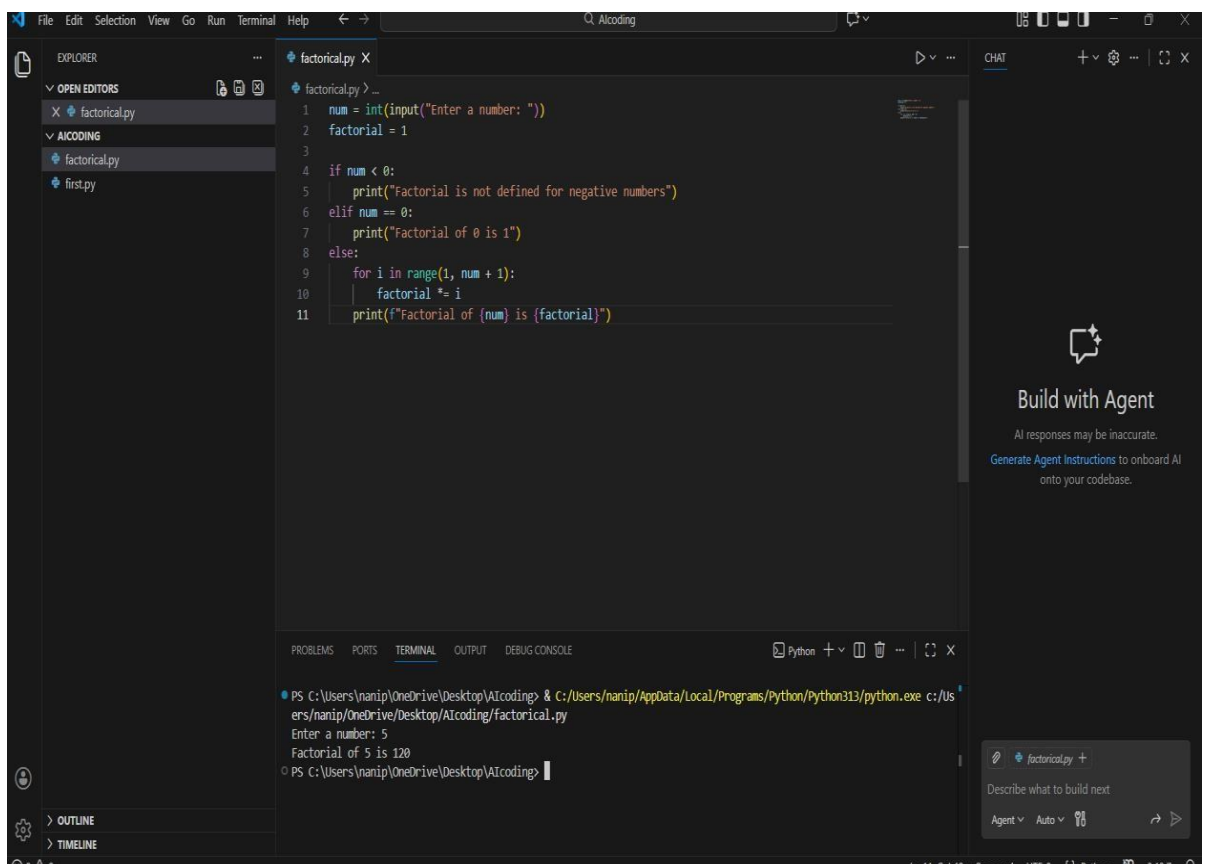




The screenshot shows the Visual Studio Code editor with a file named `factorial.py` open. The code is a Python script that calculates the factorial of a number. The code is as follows:

```
1 num = int(input("Enter a number: "))
2 factorial = 1
3
4 if num < 0:
5     print("Factorial is not defined for negative numbers")
6 elif num == 0:
7     print("Factorial of 0 is 1")
8 else:
9     for i in range(1, num + 1):
10         factorial *= i
11     print(f"Factorial of {num} is {factorial}")
```

The right sidebar shows the "CHAT" panel with a "Build with Agent" section. It includes a warning that "AI responses may be inaccurate" and a button to "Generate Agent Instructions to onboard AI onto your codebase".



The screenshot shows the Visual Studio Code editor with the same `factorial.py` file. The terminal at the bottom shows the execution of the script. The prompt is `PS C:\Users\nanip\OneDrive\Desktop\AICoding> .\factorial.py`. The output is:

```
Enter a number: 5
Factorial of 5 is 120
```

The right sidebar shows the "CHAT" panel with a "Build with Agent" section. It includes a warning that "AI responses may be inaccurate" and a button to "Generate Agent Instructions to onboard AI onto your codebase".

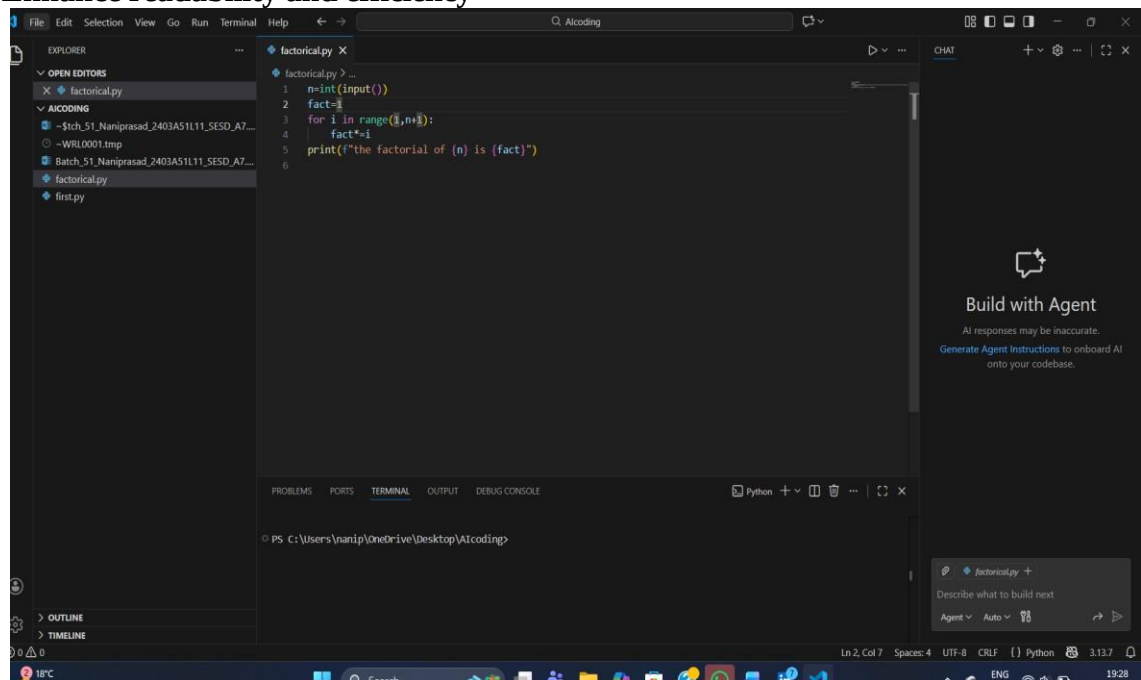
- ❖ The Copilot is very helpful because we can generate code by just giving a prompt in Copilot Chat (ctrl + I)
- ❖ The code generated was as requested in the prompt

TASK - 2

Task Description

Analyze the code generated in Task 1 and use Copilot again to:

- ❖ Reduce unnecessary variables
- ❖ Improve loop clarity
- ❖ Enhance readability and efficiency

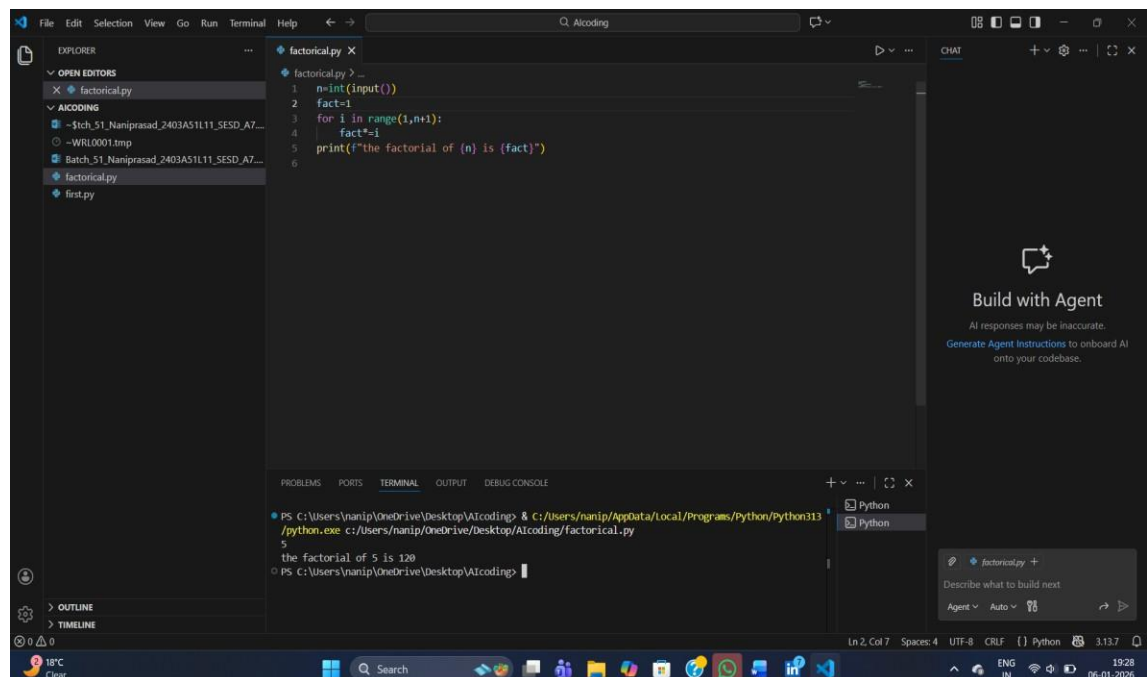


```

1 n=int(input())
2 fact=1
3 for i in range(1,n+1):
4     fact*=i
5 print(f"the factorial of {n} is {fact}")
6

```

PS C:\Users\nanip\OneDrive\Desktop\AICoding>



```

1 n=int(input())
2 fact=1
3 for i in range(1,n+1):
4     fact *= i
5 print(f"the factorial of {n} is {fact}")
6

```

PS C:\Users\nanip\OneDrive\Desktop\AICoding> & C:\Users\nanip\AppData\Local\Programs\Python\Python313\python.exe c:\Users\nanip\OneDrive\Desktop\AICoding\factorial.py

5

the factorial of 5 is 120

PS C:\Users\nanip\OneDrive\Desktop\AICoding>

What was improved?

- Shorter multiplication statement
- $\text{factorial} = \text{factorial} * i \rightarrow \text{factorial} *= i$

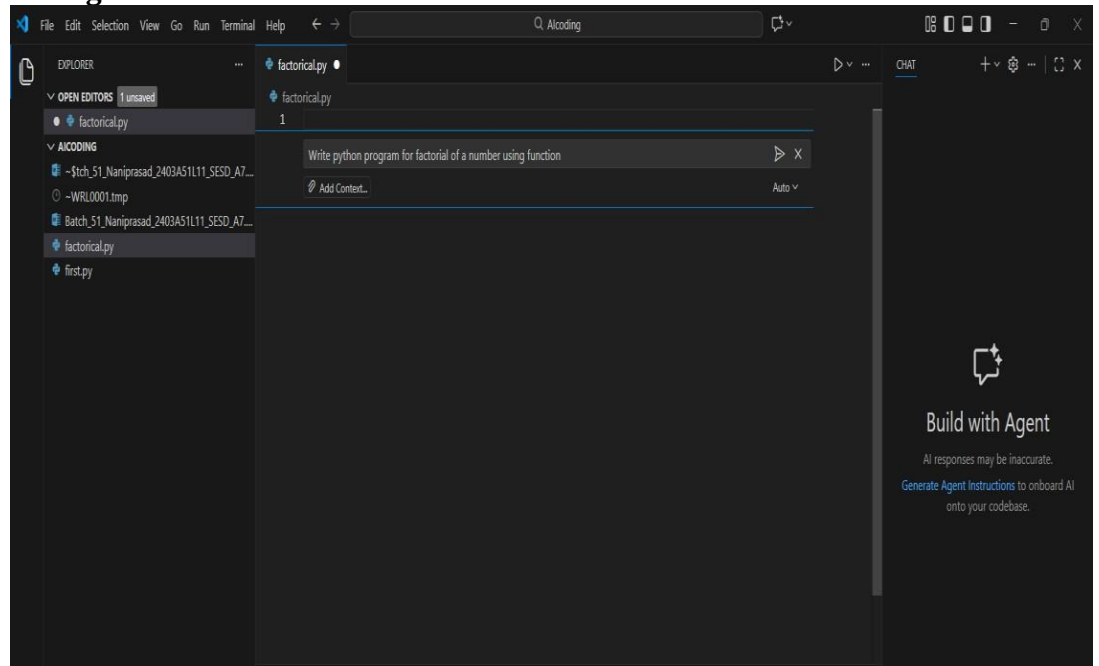
- ❖ The loop logic is self-explanatory, so the comment was removed.
- ❖ # Why the new version is better?
- ❖ Readability
 - ❖ *= is clearer and more concise.
 - Fewer lines and less clutter make the code easier to read.
- ❖ Maintainability
 - Cleaner code is easier to modify and debug.
 - Reduced redundancy lowers the chance of mistakes.
- ❖ Performance
 - Performance is effectively the same.
- ❖ *= is marginally optimized at the bytecode level, but the difference is negligible.

Task3

Task Description

Use GitHub Copilot to generate a modular version of the program by:

- ❖ Creating a user-defined function
- ❖ Calling the function from the main block



The screenshot shows the Visual Studio Code editor with a file named `factorial.py` open. The code defines a `factorial(n)` function that calculates the factorial of a number. It includes a docstring, a check for negative numbers, and a loop for calculating the factorial. The function is then used to get user input and print the result.

```

1 def factorial(n):
2     """calculate the factorial of a number."""
3     if n < 0:
4         return "Factorial is not defined for negative numbers"
5     elif n == 0 or n == 1:
6         return 1
7     else:
8         result = 1
9         for i in range(2, n + 1):
10            result *= i
11        return result
12
13 # Get input from user
14 num = int(input("Enter a number: "))
15 print(f"Factorial of {num} is: {factorial(num)}")

```

The screenshot shows the Visual Studio Code editor with the same `factorial.py` file. The terminal window at the bottom shows the command to run the script and the output for the input 5.

```

PS C:\Users\nanip\OneDrive\Desktop\AICoding> & C:/Users/nanip/AppData/Local/Programs/Python/Python313/python.exe c:/Users/nanip/OneDrive/Desktop/AICoding/factorial.py
Enter a number: 5
Factorial of 5 is: 120

```

❖ **Modularity improves reusability by:**

Allowing the factorial() function to be reused in multiple programs without rewriting code.

Making the program easier to test, update, and debug.

Improving code organization, where logic is separated from input/output handling.

Supporting scalability, as the same function can be extended or integrated into larger projects.

#Task4

Task Description

Compare the non-function and function-based Copilot-generated programs on the following criteria:

- ❖ Logic clarity
- ❖ Reusability
- ❖ Debugging ease
- ❖ Suitability for large projects
- ❖ AI dependency risk

The screenshot shows the Visual Studio Code editor with a file named `factorial.py` open. The code defines a `factorial` function and uses it to calculate the factorial of a user input. A dialog box is displayed over the code, asking to "Compare the non-function and function-based Copilot-generated programs on the following criteria:" with checkboxes for "Logic clarity", "Reusability", and "Add Context...". The terminal at the bottom shows the command `python.exe c:/Users/nanip/OneDrive/Desktop/AIcoding/factorial.py` being executed, with the output "Enter a number: 5" and "Factorial of 5 is: 120".

```

7 def factorial(n):
11     elif n == 0 or n == 1:
12         return 1
13     else:
14         result = 1
15         for i in range(2, n + 1):
16             result *= i
17         return result
18
19 # Get input from user
20 num = int(input("Enter a number: "))
21 print(f"Factorial of {num} is: {factorial(num)}")
22
23

```

Compare the non-function and function-based Copilot-generated programs on the following criteria:

- ☐ Logic clarity
- ☐ Reusability
-

Auto

```

PS C:\Users\nanip\OneDrive\Desktop\AIcoding> & C:\Users\nanip\AppData\Local\Programs\Python\Python313\python.exe c:/Users/nanip/OneDrive/Desktop/AIcoding/factorial.py
Enter a number: 5
Factorial of 5 is: 120
PS C:\Users\nanip\OneDrive\Desktop\AIcoding>

```

The screenshot shows the Visual Studio Code editor with the same `factorial.py` file. A new code block has been added, performing a "Comparison Analysis" between the two approaches. The terminal shows the same execution as the previous screenshot, but the code in the editor has been updated to include the comparison analysis.

```

22 print(f"Factorial of {num} is: {factorial(num)}")
23
24 # Comparison Analysis
25 print("\n-- Comparison of Approaches --")
26 print("Non-function approach:")
27 print("  Logic clarity: Simple but mixed concerns")
28 print("  Reusability: Low - code cannot be reused")
29 print("  Debugging ease: Harder - logic embedded in main flow")
30 print("  Large projects: Poor - code duplication likely")
31 print("  AI dependency: Moderate - straightforward logic")
32
33 print("\nFunction-based approach:")
34 print("  Logic clarity: High - isolated and documented")
35 print("  Reusability: High - can be imported and used anywhere")
36 print("  Debugging ease: Easy - logic is testable and modular")
37 print("  Large projects: Excellent - follows best practices")
38 print("  AI dependency: Lower - cleaner code requires less AI assistance")
39 print("\nRecommendation: Use function-based approach for production code")

```

```

PS C:\Users\nanip\OneDrive\Desktop\AIcoding> & C:\Users\nanip\AppData\Local\Programs\Python\Python313\python.exe c:/Users/nanip/OneDrive/Desktop/AIcoding/factorial.py
Enter a number: 5
Factorial of 5 is: 120
PS C:\Users\nanip\OneDrive\Desktop\AIcoding>

```



```

18
19
20 # Get input from user
21 num = int(input("Enter a number: "))
22 print(f"Factorial of {num} is: {factorial(num)}")
23 # Comparison Analysis
24 print("\n--- Comparison of Approaches ---")
25 print("Non-function approach:")
26 print("  Logic clarity: Simple but mixed concerns")
27 print("  Reusability: Low - code cannot be reused")
28 print("  Debugging ease: Harder - logic embedded in main flow")
29 print("  Large projects: Poor - code duplication likely")
30 print("  AI dependency: Moderate - straightforward logic")
31
32 print("\nFunction-based approach:")
33 print("  Logic clarity: High - isolated and documented")
34 print("  Reusability: High - can be imported and used anywhere")
35 print("  Debugging ease: Easy - logic is testable and modular")

```

Terminal Output:

```

PS C:\Users\nanip\OneDrive\Desktop\AICoding> & C:\Users\nanip\AppData/Local/Programs/Python/Python313/python.exe c:/Users/nanip/OneDrive/Desktop/AICoding/factorial.py

--- Comparison of Approaches ---
Non-function approach:
Logic clarity: Simple but mixed concerns
Reusability: Low - code cannot be reused
Debugging ease: Harder - logic embedded in main flow
Large projects: Poor - code duplication likely
AI dependency: Moderate - straightforward logic

Function-based approach:
Logic clarity: High - isolated and documented
Reusability: High - can be imported and used anywhere
Debugging ease: Easy - logic is testable and modular
Large projects: Excellent - follows best practices
AI dependency: Lower - cleaner code requires less AI assistance

```

TASK - 5

Task Description

Prompt Copilot to generate:

An iterative version of the logic

A recursive version of the same logic

factorial.py

```

1

```

write the python program using recursion for factorial of number

Auto

The screenshot shows the Visual Studio Code editor with a file named `factorial.py` open. The code defines a recursive function `factorial(n)` that calculates the factorial of a number. The function includes a base case for `n < 0` (raising a `ValueError`) and recursive calls for `n > 1`. A test function is also present, which prompts the user to enter a number and prints the result. The chat sidebar on the right displays the message "Build with Agent" and "AI responses may be inaccurate. Generate Agent Instructions to onboard AI onto your codebase."

```

1 def factorial(n):
2     if n < 0:
3         raise ValueError("Factorial is not defined for negative numbers")
4     elif n == 0 or n == 1:
5         return 1
6     else:
7         return n * factorial(n - 1)
8
9 # Test the function
10 if __name__ == "__main__":
11     number = int(input("Enter a number: "))
12     result = factorial(number)
13     print(f"Factorial of {number} is {result}")

```

This screenshot shows the same `factorial.py` file, but now the terminal at the bottom is active. It shows the command to run the script: `python c:\Users\nanip\OneDrive\Desktop\AICoding\factorial.py`. The user has entered the number 5, and the output is "Factorial of 5 is 120". The chat sidebar remains on the right.

```

PS C:\Users\nanip\OneDrive\Desktop\AICoding> python c:\Users\nanip\OneDrive\Desktop\AICoding\factorial.py
Enter a number: 5
Factorial of 5 is 120
PS C:\Users\nanip\OneDrive\Desktop\AICoding>

```

Explanation:

How the Function Works

1. **Negative number check**
Factorials are not defined for negative numbers. If the input is negative, the program raises an error message.
 2. **Base cases**
For 0 and 1, the factorial is defined as 1. This acts as the stopping condition for recursion.
 3. **Recursive case**
For numbers greater than 1, the function calls itself with $n-1$. This recursive process continues until it reaches the base case.
- Example:**
- To compute $5!$, the function calculates $5 \times 4!$.

- Then $4!$ becomes $4 \times 3!$, and so on, until it reaches $1!$.

- **Main Program Flow**

- The program asks the user to enter a number.
- It then calls the factorial function with that number.
- Finally, it prints the result in a clear message.

- **Example Execution**

If the user enters 5:

- The recursive calls break it down step by step until reaching 1.
- The final result is 120.

So the program outputs: *Factorial of 5 is 120.*

Summary

This program demonstrates:

- Recursion (function calling itself).
- Error handling (for negative inputs).
- Base cases (to stop recursion).
- User interaction (taking input and displaying output).

